



Conventional ML Methods

Applied Machine Learning

Potential Data Problems

Assistant Professor | **Chun-Hsiang Chan**

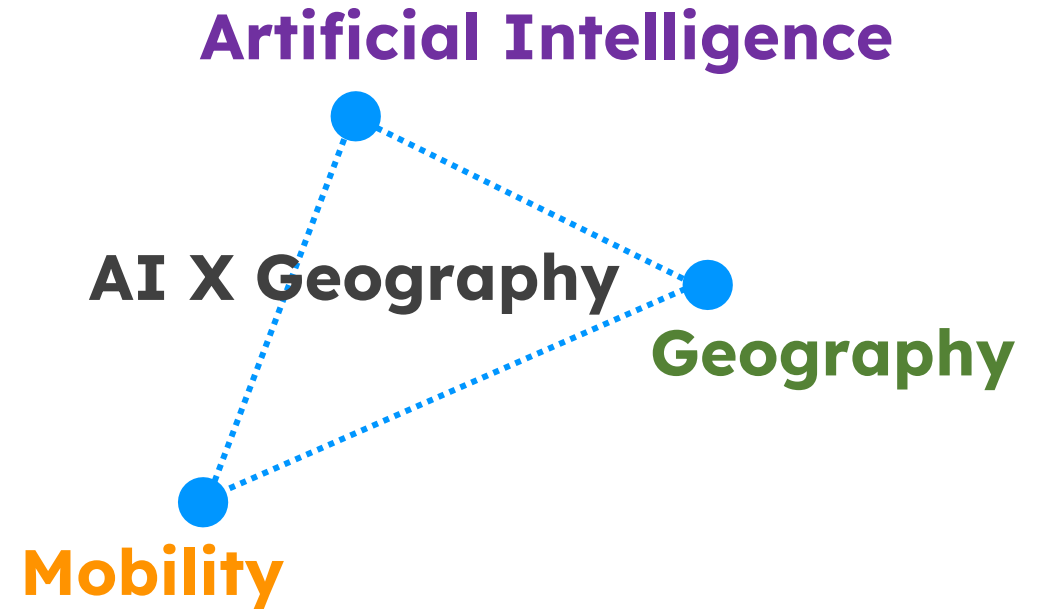
Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

- 01 Preprocessing Checklist
- 02 Evaluation Metrics for Classification
- 03 Data Imbalance
- 04 Some Practical Issues



Preprocessing Checklist

Missing Data | Unknown Blanks | Unit Alignment | Unknown Zeros | Outliers

There are a few points you should check before training your model, such as

1) Missing data: NA, NaN, na, nan, or Na

In most cases, missing values are blank in the dataset; hence, you may directly use `np.isnan` or `pd.isna` for checking all missing values.

Usually, we will remove all rows (records) which has one or more missing data in any columns (features).

Preprocessing Checklist

Missing Data | Unknown Blanks | Unit Alignment | Unknown Zeros | Outliers

1) Missing data: NA, NaN, na, nan, or Na

However, in reality, sometimes, the missing data will be performed via special number, for example in weather data, -9991 (instrument maintenance), -9996 (data aggregated in next record), -9997 (no data for unknown reason/ broken), -9998 (Trace) and -9999 (non-observation period).

As a result, you should do exploratory data analysis (EDA) before any further analyses or modeling. Sometimes, outliers are not meaningless and you should clearly understand all data definition and specification.

Preprocessing Checklist

Missing Data | Unknown Blanks | Unit Alignment | Unknown Zeros | Outliers

1) Missing data: NA, NaN, na, nan, or Na

In some cases, missing data will appear as a blank or special symbols, such as “-”, “—”, “---”, or “ ”. Therefore, missing data not only performs as special numbers, but also presents as special symbols.

When you do EDA, you have to investigate all values of each column to avoid all mistakes, leading to an incorrect or biased model.

Preprocessing Checklist

Missing Data | **Unknown Blanks** | **Unit Alignment** | Unknown Zeros | Outliers

2) Unknown Blanks or Tabs: "Taipei ", " Taipei", " Taipei "

Sometimes, we found that the data might be inserted some unexpected blanks/ tabs, appearing in the values, e.g., dictation data (medical records or questionnaire).

3) Unit Alignment:

Differences in value ranges can significantly affect the training process and convergence speed of gradient-based models because features with larger scales may produce disproportionately large gradients and dominate parameter updates.

Preprocessing Checklist

Missing Data | Unknown Blanks | Unit Alignment | **Unknown Zeros** | **Outliers**

4) Unknown Zeros:

Typically, we often consider that "zero" is just a number (e.g., rainfall), sometimes, it represents a category. For example, "0" is woman/ female subject in gender, but in some occasions, "0" is other m

5) Outliers:

In statistical analysis, we usually delete outliers from the datasets, but in the case of classification, outliers sometimes represents the rare cases (e.g., Thyroid metastasis), which might be very important to their domain.

Evaluation Metrics for Classification

Confusion Matrix | Four Metrics | ROC & AUC

Here, we introduce the evaluation metrics of classifiers.

First, we have to learn what the confusion metrics are.

Predict 1; Ground Truth 1 → True Positive

Predict 1; Ground Truth 0 → False Positive

Predict 0; Ground Truth 0 → True Negative

Predict 0; Ground Truth 1 → False Negative

Confusion Matrix		Ground Truth	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Evaluation Metrics for Classification

Confusion Matrix | **Four Metrics** | ROC & AUC

Second, there are four common quantitative metrics for classification case.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$

$$F1 - score = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}}$$

Confusion Matrix		Ground Truth	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Evaluation Metrics for Classification

$$TPR = \frac{TP}{TP + FN}; FPR = \frac{FP}{TN + FP}$$

Confusion Matrix | Four Metrics | **ROC & AUC**

Some studies prefer to show the Receiver Operating Characteristic (ROC) curve and corresponding area under curve (AUC) to demonstrate the model performance.

First of all, ROC compassed of False Positive Rate (FPR) and True Positive Rate (TPR).

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{TN + FP}$$

Evaluation Metrics for Classification

$$TPR = \frac{TP}{TP + FN}; FPR = \frac{FP}{TN + FP}$$

Confusion Matrix | Four Metrics | **ROC & AUC**

Here, we give an example to demonstrate how to calculate ROC curve and AUC.

ID	Actual (Y)	Predicted Probability
A	1	0.95
B	1	0.90
C	0	0.85
D	1	0.80
E	0	0.70
F	1	0.65
G	1	0.55
H	0	0.40
I	0	0.30
J	0	0.10

When you obtain the probability of each records, you have to rank them by probability from high to low.

of Positive: 5; # of Negative: 5

$$TPR = \frac{TP}{5}; FPR = \frac{FP}{5}$$

Each additional positive case increases the TPR by 0.2, moving the ROC curve upward.

Evaluation Metrics for Classification

$$TPR = \frac{TP}{TP + FN}; FPR = \frac{FP}{TN + FP}$$

Confusion Matrix | Four Metrics | **ROC & AUC**

ID	Actual (Y)	Predicted Probability
A	1	0.95
B	1	0.90
C	0	0.85
D	1	0.80
E	0	0.70
F	1	0.65
G	1	0.55
H	0	0.40
I	0	0.30
J	0	0.10

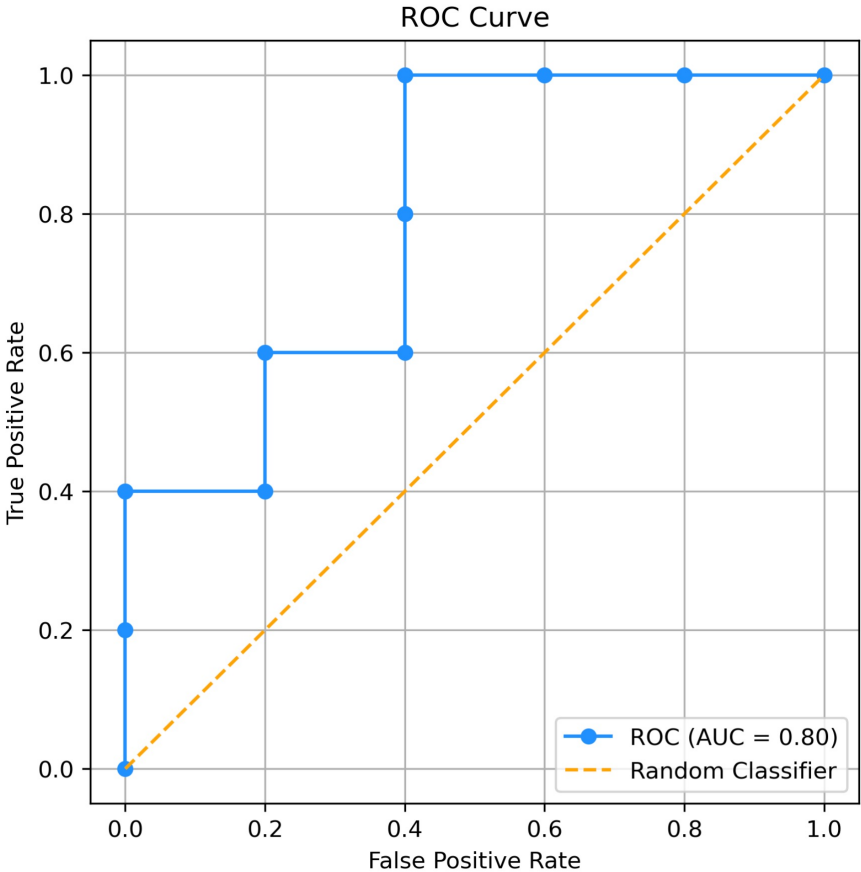
Threshold	Newly included	Actual	TP	FP	TPR	FPR
> 0.95	—	—	0	0	0.0	0.0
0.95	A	1	1	0	0.2	0.0
0.90	B	1	2	0	0.4	0.0
0.85	C	0	2	1	0.4	0.2
0.80	D	1	3	1	0.6	0.2
0.70	E	0	3	2	0.6	0.4
0.65	F	1	4	2	0.8	0.4
0.55	G	1	5	2	1.0	0.4
0.40	H	0	5	3	1.0	0.6
0.30	I	0	5	4	1.0	0.8
0.10	J	0	5	5	1.0	1.0

Evaluation Metrics for Classification

$$TPR = \frac{TP}{TP + FN}; FPR = \frac{FP}{TN + FP}$$

Confusion Matrix | Four Metrics | **ROC & AUC**

Threshold	Newly included	Actual	TP	FP	TPR	FPR
> 0.95	—	—	0	0	0.0	0.0
0.95	A	1	1	0	0.2	0.0
0.90	B	1	2	0	0.4	0.0
0.85	C	0	2	1	0.4	0.2
0.80	D	1	3	1	0.6	0.2
0.70	E	0	3	2	0.6	0.4
0.65	F	1	4	2	0.8	0.4
0.55	G	1	5	2	1.0	0.4
0.40	H	0	5	3	1.0	0.6
0.30	I	0	5	4	1.0	0.8
0.10	J	0	5	5	1.0	1.0



Evaluation Metrics for Classification

Confusion Matrix | Four Metrics | **ROC & AUC**

Area Under the ROC Curve (AUC)

AUC represents the probability that the model assigns a higher prediction score to a randomly selected positive case than to a randomly selected negative case.

AUC = 1.0: Perfect discrimination

AUC = 0.5: No discriminative ability (equivalent to random ranking)

AUC < 0.5: The model tends to rank negative cases higher than positive cases

Data Imbalance

Overview | Undersampling | Oversampling | SMOTE | Summary

In real world dataset, the number of each category is not the same, or even sometimes has a huge gap among them, such as the number of poison mushrooms is much larger than non-poison ones.

Therefore, your model might be only trained by the characteristic of poison mushroom, or even awful, your model just guess all mushrooms are poison.

Can you imagine the reason why your model cannot learn the characteristics of non-poison mushroom?

Data Imbalance

Overview | Undersampling | Oversampling | SMOTE | Summary

The key to understand this issue is **probability**; therefore, the only way to fix or we say to reduce the occurrence of this issue is to adjust the proportion of each category.

However, you cannot collect more samples from the minority (the category with small sample size) in most of time due to several reasons.

So, there are two ways to alleviate (solve, but I do not prefer to use this verb) this problem: **undersampling** for the **majority** and **oversampling** for **minority**.

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Undersampling

This solution is pretty straightforward, which reduce the number of majority (the category with larger sample size), called undersampling.

Normally we may adjust the proportion of majority to minority from 10:1 to 3:1, 2:1, or even 1:1, with random selection.

Here we have a simple experiment to tell you the impact of sample size adjustment on model performance.

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# import packages
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt

# load data
data_df = pd.read_csv('mammography.csv', header=None)

# preview data
data_df.head(5)
```

	0	1	2	3	4	5	6
0	0.230020	5.072578	-0.276061	0.832444	-0.377866	0.480322	'-1'
1	0.155491	-0.169390	0.670652	-0.859553	-0.377866	-0.945723	'-1'
2	-0.784415	-0.443654	5.674705	-0.859553	-0.377866	-0.945723	'-1'
3	0.546088	0.131415	-0.456387	-0.859553	-0.377866	-0.945723	'-1'
4	-0.102987	-0.394994	-0.140816	0.979703	-0.377866	1.013566	'-1'

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# preprocessing
data_df[6] = data_df[6].replace("-1", 0)
data_df[6] = data_df[6].replace("1", 1)
data_df[6] = data_df[6].astype(int)
# label distribution
cnt = data_df.groupby([6]).count().reset_index()[0]
cnt
```

```
0      10923
1         260
Name: 0, dtype: int64
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

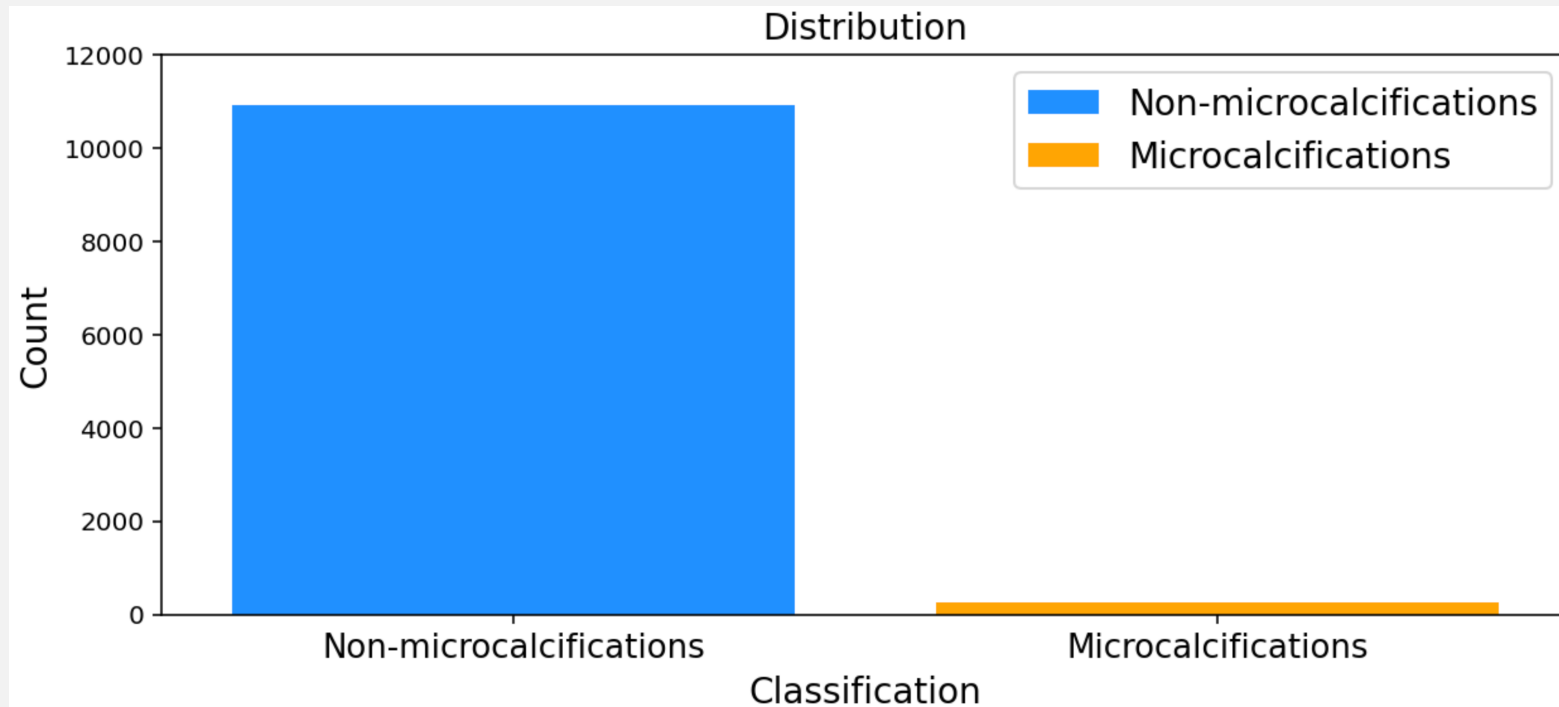
```
# visualization for label distribution
plt.figure(figsize=[10,4], dpi=144)
plt.bar(cnt.index, cnt, color=['dodgerblue','orange'],
        label=['Non-microcalcifications','Microcalcifications'])
plt.legend(fontsize=14)
plt.title('Distribution', fontsize=14)
plt.xlabel('Classification', fontsize=14)
plt.xticks(cnt.index, ['Non-microcalcifications','Microcalcifications'], fontsize=13)
plt.ylabel('Count', fontsize=14)
plt.axis([-0.5, 1.5, 0, 12000])
plt.grid(True)
plt.show()
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

visualization for label distribution



Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

model training part

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
import sklearn.metrics as skm
```

```
import joblib
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
def model_training(X_train, y_train, save_dir, name):  
    if os.path.isfile(f'{save_dir}/dvc_'+name+'.joblib'):  
        print('loading existing DecisionTreeClassifier model ...')  
        best_dtc = joblib.load(f'{save_dir}/dvc_'+name+'.joblib')  
    else:  
        print('no existing DecisionTreeClassifier model found, training a new one...')  
        param_grid_dtc = {'class_weight': ['balanced'], 'criterion': ['gini', 'entropy'],  
                           'max_depth': [3, 5, 7, 10], 'max_features': [None, 0.7, 0.8],  
                           'min_samples_split': [10, 20, 40],  
                           'min_samples_leaf': [5, 10, 20]}
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
dtc = DecisionTreeClassifier(random_state=42)
dtc_grid = GridSearchCV(estimator=dtc,
                        param_grid=param_grid_dtc,
                        scoring='f1_weighted',
                        cv=10,
                        n_jobs=-1,
                        verbose=0)

dtc_grid.fit(X_train, y_train)
dtc_grid.best_params_
dtc_grid.best_score_
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# save the best GradientBoostingClassifier model
best_dtc = dtc_grid.best_estimator_
joblib.dump(best_dtc, f'{save_dir}/dtc_'+name+'.joblib')
return best_dtc
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
def test_procedure(X_train, X_test, y_train, y_test, model_obj):  
    y_train_pred = model_obj.predict(X_train)  
    y_train_pred_proba = model_obj.predict_proba(X_train)[:, 1]  
    y_test_pred = model_obj.predict(X_test)  
    y_test_pred_proba = model_obj.predict_proba(X_test)[:, 1]  
  
    fpr, tpr, thresholds = skm.roc_curve(y_test, y_test_pred_proba)  
    roc_auc = skm.auc(fpr, tpr)  
    print(f'ROC AUC: {roc_auc:.4f}')
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

scores

```
print('===== train =====')
print('precision:', np.round(skm.precision_score(y_train, y_train_pred), 3))
print('recall:', np.round(skm.recall_score(y_train, y_train_pred), 3))
print('f1-score:', np.round(skm.f1_score(y_train, y_train_pred), 3))
print('accuracy:', np.round(model_obj.score(X_train, y_train), 3))
print('===== test =====')
print('precision:', np.round(skm.precision_score(y_test, y_test_pred), 3))
print('recall:', np.round(skm.recall_score(y_test, y_test_pred), 3))
print('f1-score:', np.round(skm.f1_score(y_test, y_test_pred), 3))
print('accuracy:', np.round(model_obj.score(X_test, y_test), 3))
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
train_metrics = [skm.precision_score(y_train, y_train_pred),  
                 skm.recall_score(y_train, y_train_pred),  
                 skm.f1_score(y_train, y_train_pred),  
                 model_obj.score(X_train, y_train), roc_auc],  
test_metrics = [skm.precision_score(y_test, y_test_pred),  
                skm.recall_score(y_test, y_test_pred),  
                skm.f1_score(y_test, y_test_pred),  
                model_obj.score(X_test, y_test), roc_auc]  
return fpr, tpr, train_metrics, test_metrics
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# set path for models
save_dir = '/Users/toodou/Documents/python/ML/models'
# fetch X and y data
y_binary = data_df[6]
x_features = data_df.iloc[:,6]
# prepare training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(x_features, y_binary,
                                                    test_size=0.3, random_state=42)
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# model training
```

```
best_dtc = model_training(X_train, y_train, save_dir, 'original')
```

```
# show the best model parameters and scores
```

```
best_dtc.get_params()
```

```
loading existing DecisionTreeClassifier model ...
{'ccp_alpha': 0.0,
 'class_weight': 'balanced',
 'criterion': 'entropy',
 'max_depth': 10,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 5,
 'min_samples_split': 10,
 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None,
 'random_state': 42,
 'splitter': 'best'}
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# perform all metrics
```

```
fpr, tpr, train_metrics, test_metrics = test_procedure(X_train, X_test, y_train,  
                                                       y_test, best_dtc)
```

```
ROC AUC: 0.8318
```

```
===== train =====
```

```
precision: 0.607
```

```
recall: 0.973
```

```
f1-score: 0.748
```

```
accuracy: 0.984
```

```
===== test =====
```

```
precision: 0.362
```

```
recall: 0.73
```

```
f1-score: 0.484
```

```
accuracy: 0.966
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# select all minority and majority
minor = data_df[data_df[6]==1].reset_index(drop=True)
majority = data_df[data_df[6]==0].reset_index(drop=True)
# randomly oversampling with replacement
majority20 = majority.sample(frac=0.8)
# concatenate all
mU20 = pd.concat([majority20, minor], axis=0).reset_index(drop=True)
# preview dataset
mU20.tail()
```

	0	1	2	3	4	5	6
8993	-0.250012	-0.377300	-0.321142	1.269157	3.652984	1.092791	1
8994	0.281343	-0.417112	-0.366224	0.851010	2.789649	1.345700	1
8995	1.204988	1.763724	-0.501468	1.562408	6.489072	0.931294	1
8996	0.736644	-0.222474	-0.050653	1.509665	0.539269	1.315229	1
8997	0.177003	-0.191508	-0.501468	1.578864	7.750705	1.555951	1

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# fetch X and y data
y_binary = mU20[6]
x_features = mU20.iloc[:,6]

# prepare training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(x_features, y_binary,
                                                    test_size=0.3, random_state=42)
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# model training
```

```
best_dtc_u20 = model_training(X_train, y_train, save_dir, 'Under20')
```

```
# show the best model parameters and scores
```

```
best_dtc_u20.get_params()
```

```
# perform all metrics
```

```
fpr_u20, tpr_u20, train_metrics_u20, test_metrics_u20 = test_procedure(X_train,  
X_test, y_train, y_test,  
best_dtc_u20)
```

```
no existing DecisionTreeClassifier model found, training a new one...  
ROC AUC: 0.7845  
===== train =====  
precision: 0.625  
recall: 0.964  
f1-score: 0.759  
accuracy: 0.984  
===== test =====  
precision: 0.545  
recall: 0.663  
f1-score: 0.598  
accuracy: 0.97
```

Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

```
# ROC and AUC
```

```
plt.figure(figsize=(10, 6), dpi=300)
```

```
plt.plot(fpr, tpr, color='orange', label=f'[Original] ROC curve (area =  
{test_metrics[4]:.2f})')
```

```
plt.plot(fpr_u20, tpr_u20, color='skyblue', label=f'[20% Undersampling] ROC  
curve (area = {test_metrics_u20[4]:.2f})')
```

```
plt.plot(fpr_u50, tpr_u50, color='dodgerblue', label=f'[50% Undersampling] ROC  
curve (area = {test_metrics_u50[4]:.2f})')
```

```
plt.plot(fpr_u80, tpr_u80, color='navy', label=f'[80% Undersampling] ROC curve  
(area = {test_metrics_u80[4]:.2f})')
```

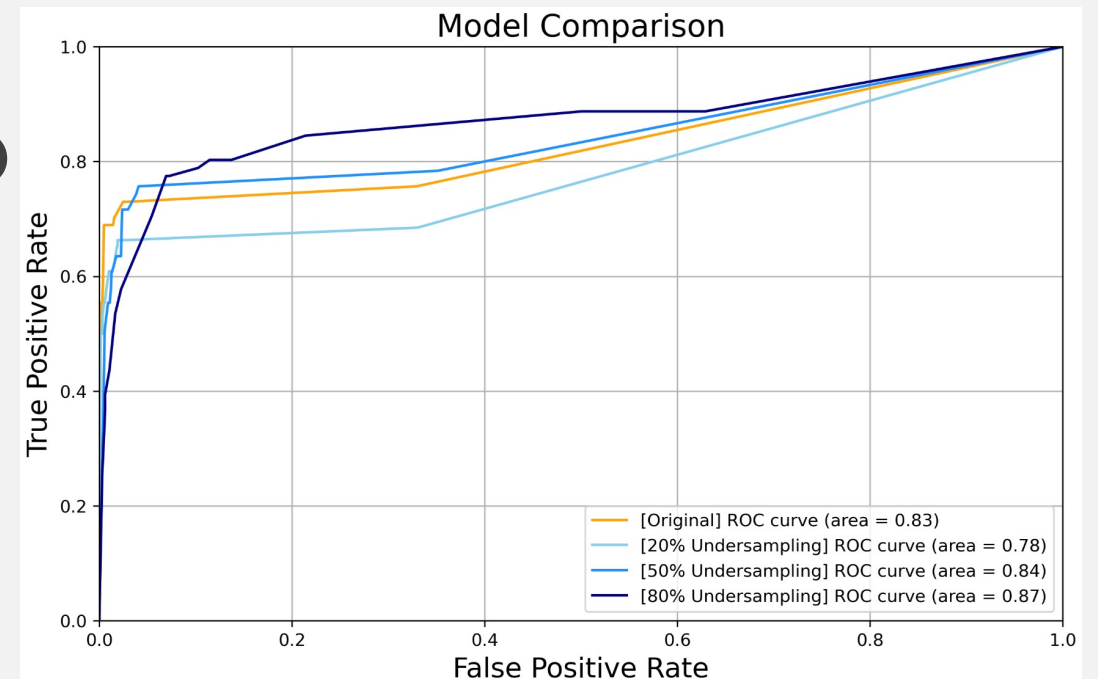
Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

Experiment #1 Undersampling for Majority

ROC and AUC

```
plt.title('Model Comparison', fontsize=18)
plt.legend(loc='lower right')
plt.xlabel('False Positive Rate', fontsize=16)
plt.ylabel('True Positive Rate', fontsize=16)
plt.axis([0,1,0,1])
plt.grid(True)
plt.show()
```



Data Imbalance

Overview | **Undersampling** | Oversampling | SMOTE | Summary

In reality, we cannot adjust the sample size to better the model training process and model performance.

- 1) The **sample size of minority is too small**
- 2) The **variation/ diversity of majority is large**

So, if we cannot deal with the majority, then we have to think about how to enlarge the sample size of the minority without collecting more data.

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Oversampling

At the beginning, we may try the most conventional method, that is minority oversampling with replacement.

How do you think about this solution?

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

```
# here, we demonstrate 1-fold-increase oversampling method
# you may try 2-, 5-, 10-, or 20-fold-increase oversampling to observe the effects on
# model performance
# select all minority and majority
minor = data_df[data_df[6]==1].reset_index(drop=True)
majority = data_df[data_df[6]==0].reset_index(drop=True)
# randomly oversampling with replacement
minor1 = minor.iloc[np.random.randint(0, len(minor), size=130)]
```

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

```
# concatenate original minority and oversampled minority
m1 = pd.concat([minor, minor1], axis=0).reset_index(drop=True)
# concatenate all
Mm1 = pd.concat([majority, m1], axis=0).reset_index(drop=True)
# preview dataset
Mm1.tail()
```

	0	1	2	3	4	5	6
11308	1.108609	-0.142849	-0.140816	3.797447	0.468941	1.260381	1
11309	2.377287	0.525115	-0.546550	2.019373	10.939259	1.001377	1
11310	0.215791	-0.178238	-0.411305	1.491099	-0.377866	1.507197	1
11311	-0.029306	-0.355182	-0.366224	-0.859553	-0.377866	-0.945723	1
11312	-0.096889	-0.434807	-0.095734	0.951854	1.958943	1.421878	1

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

```
# fetch X and y data
y_binary = Mm1[6]
x_features = Mm1.iloc[:,6]
# prepare training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(x_features, y_binary,
                                                    test_size=0.3, random_state=42)
# model training
best_dtc1 = model_training(X_train, y_train, save_dir, '1fold')
# perform all metrics
fpr1, tpr1, train_metrics1, test_metrics1 = test_procedure(X_train, X_test, y_train,
                                                            y_test, best_dtc1)
```

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

model performance

```
no existing DecisionTreeClassifier model found, training a new one...
ROC AUC: 0.9090
===== train =====
precision: 0.625
recall: 0.977
f1-score: 0.762
accuracy: 0.98
===== test =====
precision: 0.581
recall: 0.857
f1-score: 0.692
accuracy: 0.972
```

Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

```
# ROC and AUC
```

```
plt.figure(figsize=(10, 6), dpi=300)
```

```
plt.plot(fpr, tpr, color='orange', label=f'[Original] ROC curve (area =  
{test_metrics[4]:.2f})')
```

```
plt.plot(fpr1, tpr1, color='skyblue', label=f'[1-fold Oversampling] ROC curve (area  
= {test_metrics1[4]:.2f})')
```

```
plt.plot(fpr2, tpr2, color='dodgerblue', label=f'[2-fold Oversampling] ROC curve  
(area = {test_metrics2[4]:.2f})')
```

```
plt.plot(fpr10, tpr10, color='navy', label=f'[10-fold Oversampling] ROC curve  
(area = {test_metrics10[4]:.2f})')
```

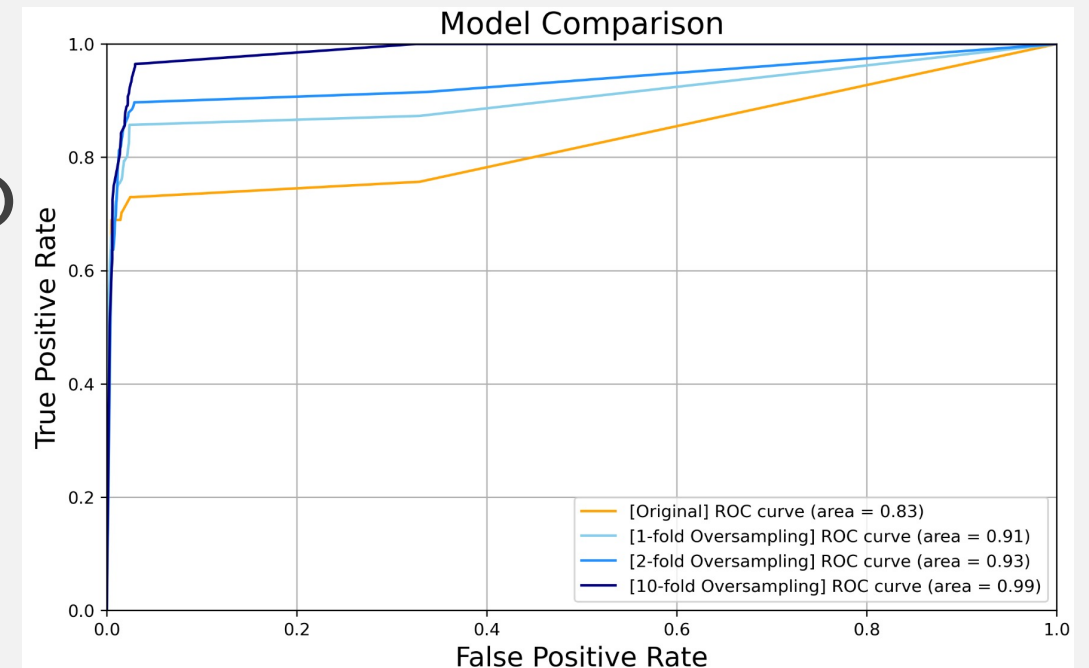
Data Imbalance

Overview | Undersampling | **Oversampling** | SMOTE | Summary

Experiment #2 Oversampling for Minority with Replacement

ROC and AUC

```
plt.title('Model Comparison', fontsize=18)
plt.legend(loc='lower right')
plt.xlabel('False Positive Rate', fontsize=16)
plt.ylabel('True Positive Rate', fontsize=16)
plt.axis([0,1,0,1])
plt.grid(True)
plt.show()
```



Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Synthetic Minority Over-sampling Technique (SMOTE)

Here, we introduce an sample generation method, called **Synthetic Minority Over-sampling Technique (SMOTE)**. Technically, this method conduct a linear interpolation with arbitrary two data points based on Gaussian distribution.

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #3 Oversampling for Minority with SMOTE

```
# import packages
from imblearn.over_sampling import SMOTE
# fetch X and y data
y_binary = data_df[6]
x_features = data_df.iloc[:,6]
# set SMOTE parameters
sm02 = SMOTE(sampling_strategy=0.2,random_state=42)
# fit SMOTE
X_res02, y_res02 = sm02.fit_resample(x_features, y_binary)
# check ratio
print(np.round(len(X_res02)/len(x_features), 4))
```

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #3 Oversampling for Minority with SMOTE

```
# concatenate original minority and oversampled minority
xX = pd.concat([x_features, X_res02], axis=0).reset_index(drop=True)
# concatenate all
yY = pd.concat([y_binary, y_res02], axis=0).reset_index(drop=True)
# prepare training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(xX, yY, test_size=0.3,
                                                    random_state=42)

# model training
best_dtc_smote02 = model_training(X_train, y_train, save_dir, 'smote_02fold')
```

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #3 Oversampling for Minority with SMOTE

```
# perform all metrics
```

```
fpr_smote02, tpr_smote02, train_metrics_smote02, test_metrics_smote02 =  
    test_procedure(X_train, X_test, y_train, y_test, best_dtc_smote02)
```

```
no existing DecisionTreeClassifier model found, training a new one...
```

```
ROC AUC: 0.9706
```

```
===== train =====
```

```
precision: 0.843
```

```
recall: 0.97
```

```
f1-score: 0.902
```

```
accuracy: 0.979
```

```
===== test =====
```

```
precision: 0.811
```

```
recall: 0.935
```

```
f1-score: 0.869
```

```
accuracy: 0.972
```

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #3 Oversampling for Minority with SMOTE

```
# ROC and AUC
```

```
plt.figure(figsize=(10, 6), dpi=300)
```

```
plt.plot(fpr, tpr, color='orange', label=f'[Original] ROC curve (area =  
{test_metrics[4]:.2f})')
```

```
plt.plot(fpr_smote02, tpr_smote02, color='skyblue', label=f'[0.2-fold SMOTE]  
ROC curve (area = {test_metrics_smote02[4]:.2f})')
```

```
plt.plot(fpr_smote05, tpr_smote05, color='dodgerblue', label=f'[0.5-fold  
SMOTE] ROC curve (area = {test_metrics_smote05[4]:.2f})')
```

```
plt.plot(fpr_smote1, tpr_smote1, color='navy', label=f'[1-fold SMOTE] ROC  
curve (area = {test_metrics_smote1[4]:.2f})')
```

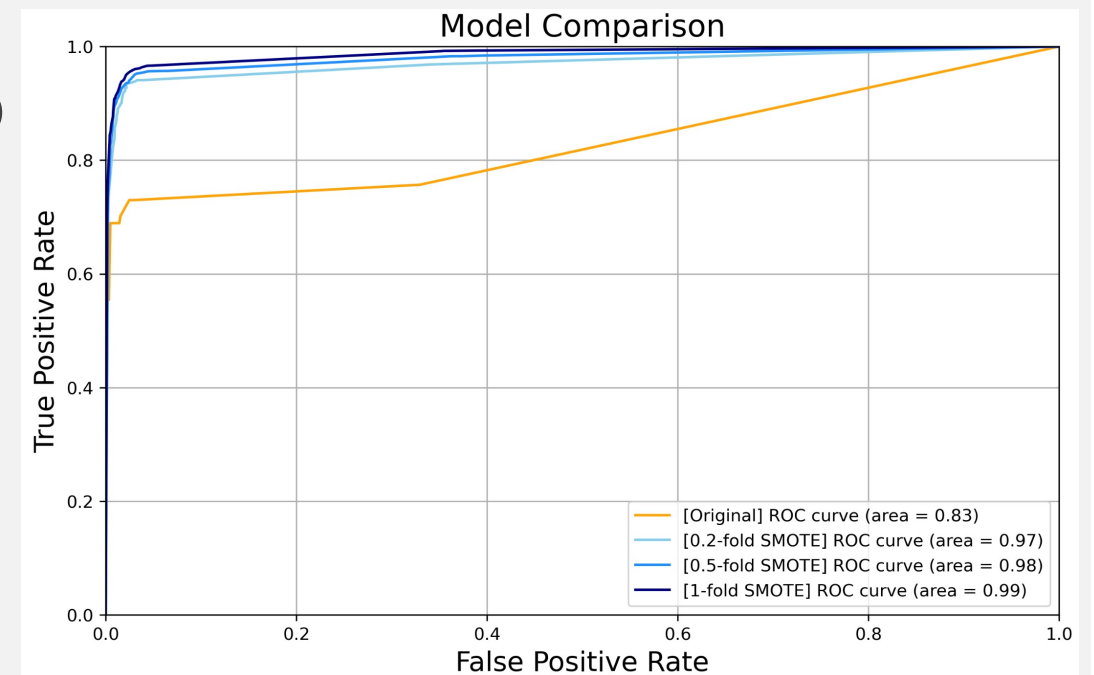
Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #3 Oversampling for Minority with SMOTE

ROC and AUC

```
plt.legend(loc='lower right')
plt.xlabel('False Positive Rate', fontsize=16)
plt.ylabel('True Positive Rate', fontsize=16)
plt.axis([0,1,0,1])
plt.grid(True)
plt.show()
```



Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

You may think that the Gaussian distribution sampling for arbitrary two data points is not reasonable in most cases. Hence, there are several variant SMOTE algorithms, such as ...

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

But can SMOTE solve everything?

...

...

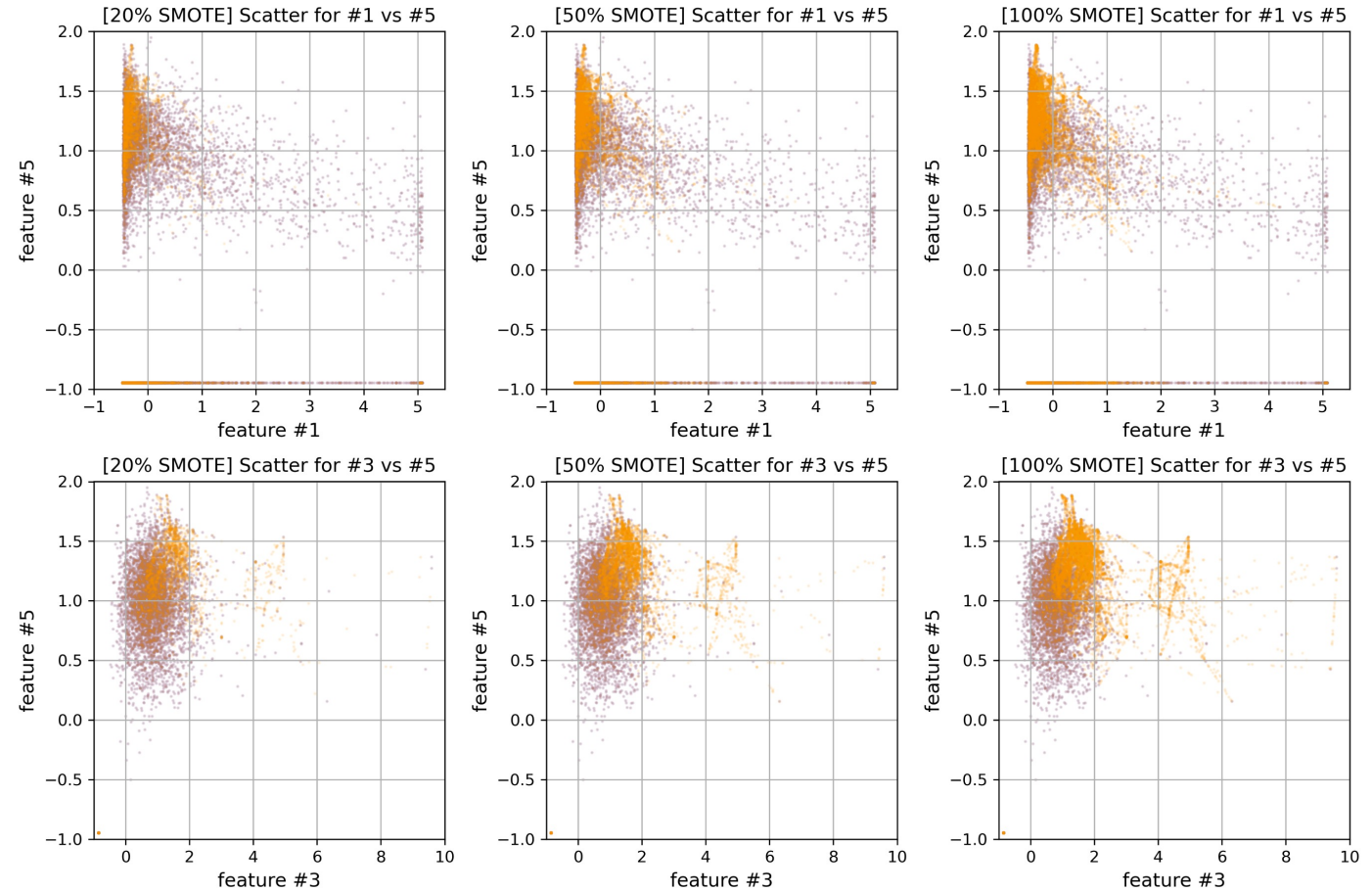
...

In the next slide, we give you an example for the potential risks/ failure of SMOTE.

Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

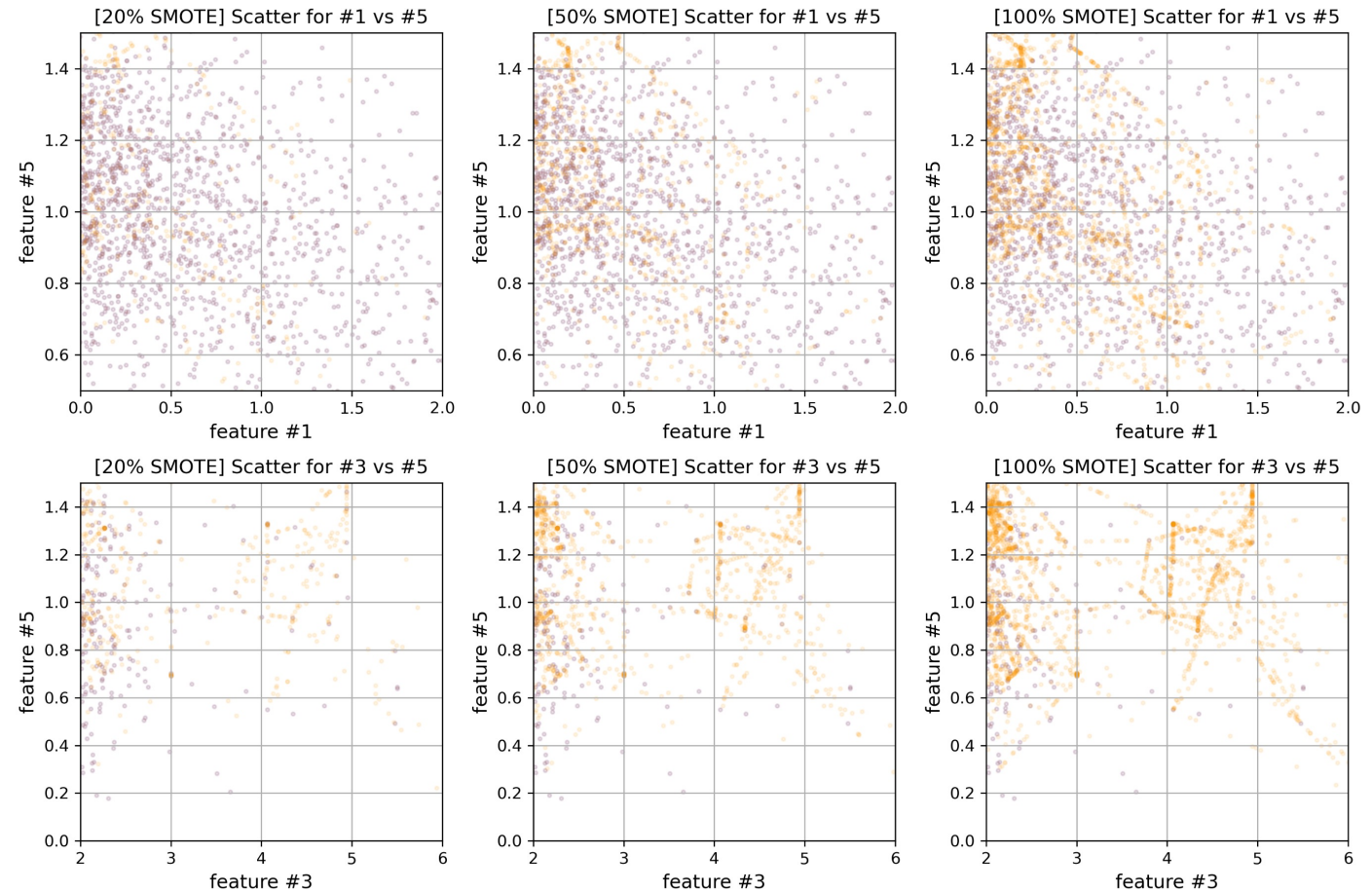
Experiment #4 SMOTE Issues



Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #4 SMOTE Issues

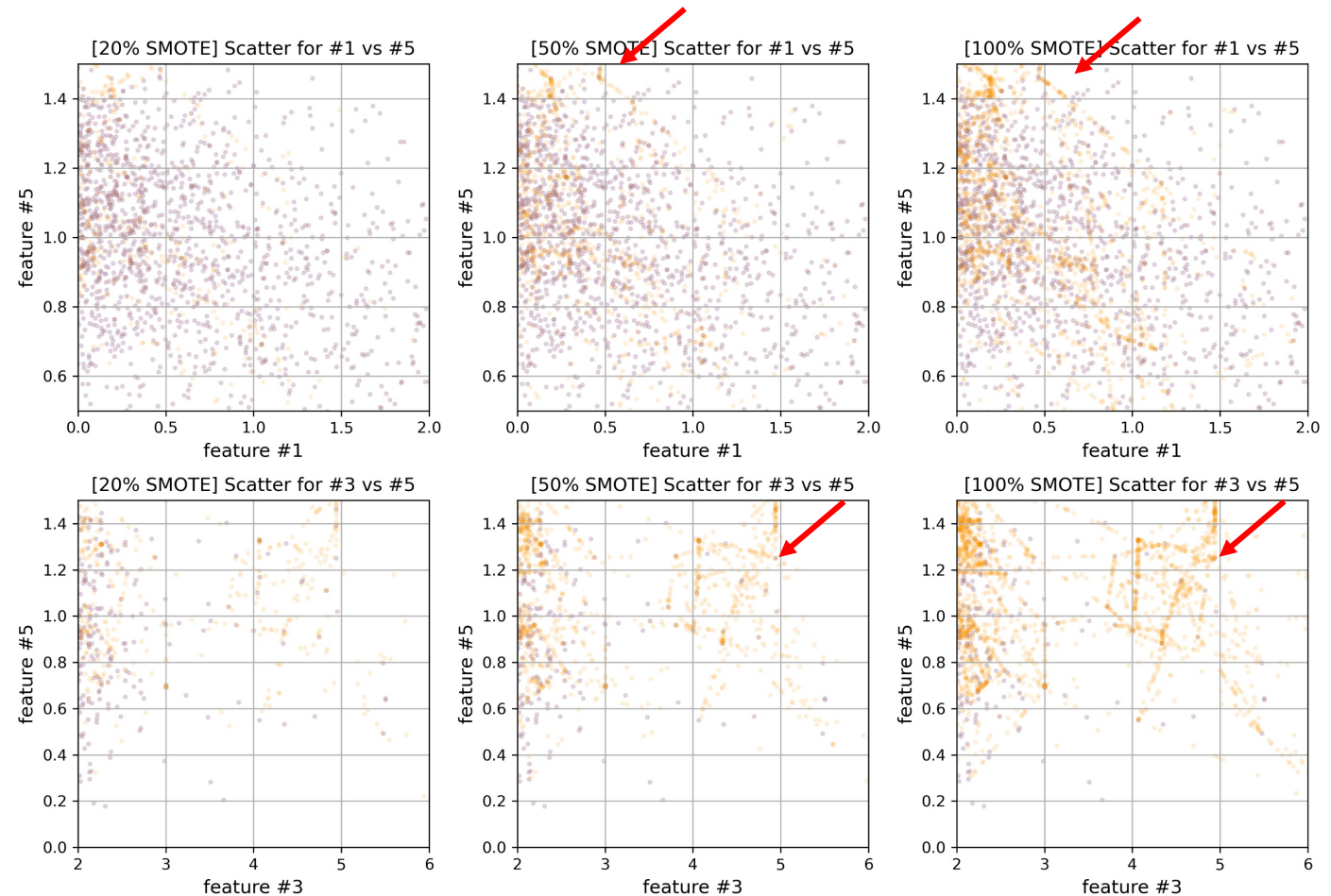


Data Imbalance

Overview | Undersampling | Oversampling | **SMOTE** | Summary

Experiment #4 SMOTE Issues

Several linear
lines appear ...



Data Imbalance

Overview | Undersampling | Oversampling | SMOTE | **Summary**

Summary

Data imbalance is a serious problem in model training process and it is hard to solve this dilemma via down sampling or SMOTE. Because these two methods might change the data distribution (patterns) from the population (which we never know, but could be tested by external testing datasets).

Besides, in some cases, ji

Some Practical Issues

Oversampling or Undersampling Issues | Data Imputation Issues |
Other Issues

Oversampling or Undersampling Issues

- 1) Medical Studies
- 2) Geographic Studies
- 3) Sociological Studies
- 4) Computer Science Studies

→ **Silver Standard:** depending on your data variation, diversity, consistency (Chan, 2026).

Some Practical Issues

Oversampling or Undersampling Issues | **Data Imputation Issues** | Other Issues

Data Imputation Issues

If you face several unknown or well-known missing data, what is your next step?

- a) Delete the records with one or more missing data
- b) Impute all values with neighboring records

If you choose (a), then you probably need to face lack of data issue for model training since most of records are lack of one or few values in some columns (maybe only one or two, even five).

Some Practical Issues

Oversampling or Undersampling Issues | **Data Imputation Issues** | Other Issues

Data Imputation Issues

In this situation, I recommend you to try:

- a1) several missing values concentrate in one or few columns, then you may consider just to delete these columns
- a2) only a few proportion of missing values in some columns, maybe you may consider to delete these records.
- a3) several missing values distribute in several columns, then maybe you need to understand the process of data collection before you do anything

Some Practical Issues

Oversampling or Undersampling Issues | **Data Imputation Issues** |
Other Issues

Data Imputation Issues

If you choose (b) imputation:

- b1) the proportion of imputation should be reviewed
- b2) you have to consider the suitability of imputation in your case
- b3) how do you define the neighboring records for imputation

Some Practical Issues

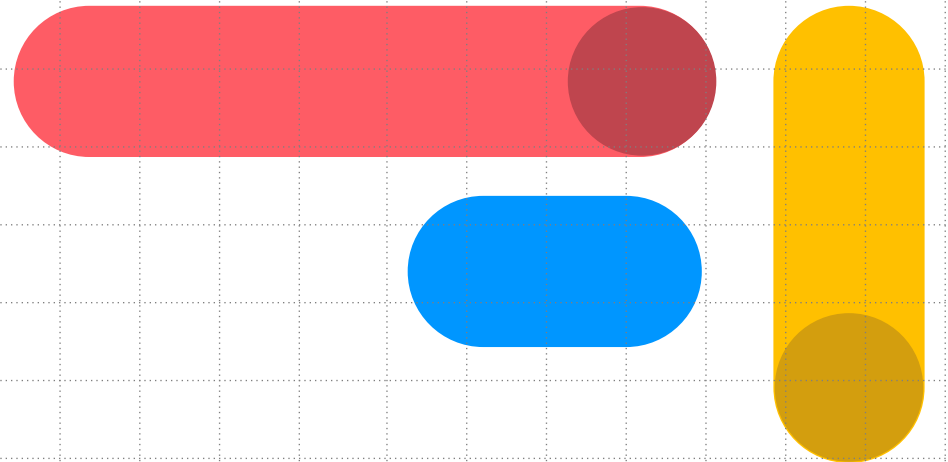
Oversampling or Undersampling Issues | Data Imputation Issues |
Other Issues

Other Issues

In data preprocessing, we may face several potential problems, especially in your final projects.

Real world data are often consisted of missing data, incorrect values, noises, wrong representation, and other mysterious problems.

All you have to do is to investigate your data collection, data/ value definition, unit, distribution, correlation, and missing data before you do EDA and model training.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: <https://toodou.github.io/>